Agentic Project Management

An enhancement of the emerging paradigm of
Spec-Driven Development

Research Study Participation Guide

Sub-Study A: Cursor

Conducted by:
Konstantinos Chaidemenos - CobuterMan

Institution:
National and Kapodistrian University of Athens



November 2025

1. Welcome & Introduction

Thank you for participating in this research study. Your participation is essential for evaluating how effectively modern AI-powered development tools support Spec-Driven Development (SDD) workflows.

The goal of this study is to compare how different AI-assisted frameworks perform on a practical, well-defined coding task. Please be assured that **you are not being tested**; the AI tools are. All data, including incomplete or non-functional code, is valuable to this research.

Data Privacy & Anonymity

All data that you will submit will be handled with strict confidentiality. Your name or any personal identifiers will not be linked to the data. The research artifacts you will provide will be used exclusively for this academic research and will be anonymized in any resulting publications. You have been assigned a unique participant ID for this study, which you must include in your submission file names as described in the submission instructions. By proceeding, you consent to the anonymous collection and use of this data as described.

2. Study Overview

2.1. Your Task

Your task is to research an existing legacy PHP codebase and implement a new feature in an existing Python codebase based on your findings. You will receive a detailed **Product Requirements Document (PRD)** as the sole source of truth for the specification, guiding both you and the AI tools.

2.2. Your Tools

You will complete this task **twice**, each time using a different AI-coding framework (e.g., APM, Spec-kit, Cursor's Plan Mode). Sessions will be **time-boxed and isolated on separate branches**. Between sessions, you will submit your code changes, chat logs, and a brief survey.

> **Learning Effects Note**
>
> We acknowledge that performing the task twice may introduce learning effects. To mitigate this, some participants will begin with Tool A, others with Tool B. This counterbalancing aims to ensure fair assessments across the study.

2.3. Your Time

Each session is strictly time-boxed to **three (3) hours**. It is important that you stop at the 3-hour mark, regardless of whether you are finished. Committing unfinished work is a perfectly valid and expected outcome.

2.4. Your Environment

To ensure the two sessions are completed in a controlled environment, set up your environment according to the following specifications:

- **Operating System:** POSIX-compliant systems such as Linux (e.g., Ubuntu 22.04+) or macOS (latest versions). For Windows, we recommend using Windows Subsystem for Linux (WSL) with a recent Ubuntu distribution.
- **Workspace:** A dedicated directory with the following repositories cloned:
  - `eclass-mcp-server` (the main codebase where you will implement the new feature). Install dependencies using:

    ```
    cd eclass-mcp-server
    uv sync --dev --all-extras
    ```

  - `openeclass` (the legacy PHP codebase where you will conduct research)
- **Python Version: 3.10+** This will include all of its dependencies.
- **IDE: Cursor 2.0+**

> **Important Cursor Access Note**
>
> This study requires access to Cursor's premium AI features. If you do not have a Pro account, you can sign up for a **7-day free trial**. Please be aware that activating this trial **requires you to enter credit card information**.
>
> Please ensure you have a **Cursor Pro or higher-tier account** before starting your sessions.

2.5. What You Will Deliver

At the end of each session, you will submit:

- A **submission zip file** containing a code patch and AI interaction logs. For example, `id_X_tool_a_submission.zip` for Tool A and `id_X_tool_b_submission.zip` for Tool B (where X is your participant ID). See Section 4.2 for instructions on creating the patch file and Section 5 for exporting AI interaction logs from Cursor.
- A brief **post-session survey** about your experience using each AI-coding framework. See Sections 4.2 and 4.3 for links to the surveys.

3. AI Tools: Setup & Instructions

Before starting your sessions, familiarize yourself with your assigned tools. Below are setup instructions and usage guidelines for each tool.

3.1. APM in Cursor

Description:

Agentic Project Management (APM) is designed to enhance your development process by decomposing specifications into actionable tasks and coordinating their execution across multiple AI agents; aiming to improve efficiency and context management.

Setup Instructions:

- Ensure you have `npm` installed on your system.
- Install the `agentic-pm` package globally using the following command:

```
npm install -g agentic-pm
```

- Once installed, use the `apm init` command within your working directory's root (alongside the other codebases) and select `Cursor`.

Familiarization:

To get acquainted with APM, visit the official documentation which includes video tutorials and getting started guides before starting your timed session.

3.2. Spec-kit in Cursor

Description:

Spec-kit is another AI-powered development framework designed to enhance the specification and planning stages of software development, ensuring a spec-first approach throughout the implementation process.

Setup Instructions:

- Ensure `uv` has been installed and set up on your system.
- Install the `specify-cli` using their official recommended installation command:

```
uv tool install specify-cli --from git+https://github.com/github/spec-kit.git
```

- Once installed, use the `specify init <your-project-name>` command within your working directory's root (alongside the other codebases) and select `Cursor`.

Familiarization:

To get acquainted with Spec-kit watch this tutorial and/or read the official documentation before starting your timed session.

3.3. Plan Mode in Cursor

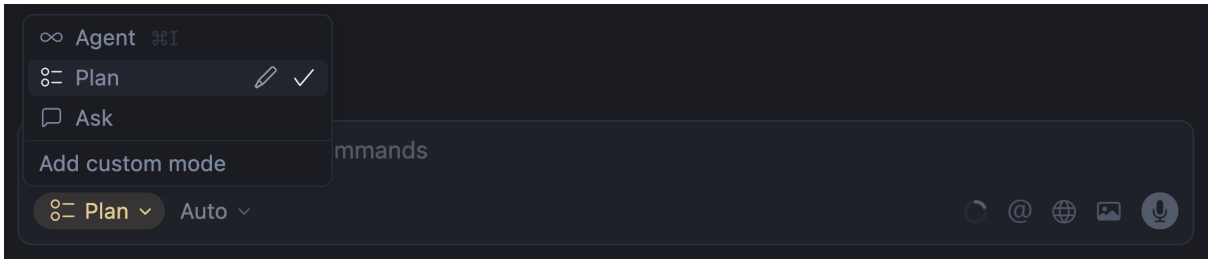
Description:

Cursor's **Plan Mode** offers a streamlined implementation of Spec-Driven Development principles by emphasizing a "first-plan-then-execute" workflow. It enables your AI assistant to break down tasks into manageable steps before execution.

After your plan has been finalized in Plan Mode, you may switch to Agent Mode if you need to further iterate on task execution. Alternatively, you can save the plan locally from Plan Mode and start a new chat in Agent Mode to execute it there.

Setup Instructions:

No additional setup is required beyond having Cursor 2.0+ installed. To enable Plan Mode simply select it in the chat mode modal:



Familiarization:

Plan Mode is integrated directly into Cursor. Use Cursor as you would normally use your AI assistant but ensure you have selected **Plan Mode**. For a brief overview, see Cursor's official introduction video.

> **Important Familiarization Note**
>
> Familiarization with your assigned tool is **crucial** for effective participation. Take time to watch provided tutorials, review documentation, and practice on unrelated tasks to build confidence. **Avoid using the actual task or codebase during practice to minimize learning effects between sessions.** The post-session survey will ask about your prior experience and familiarity with the tool.

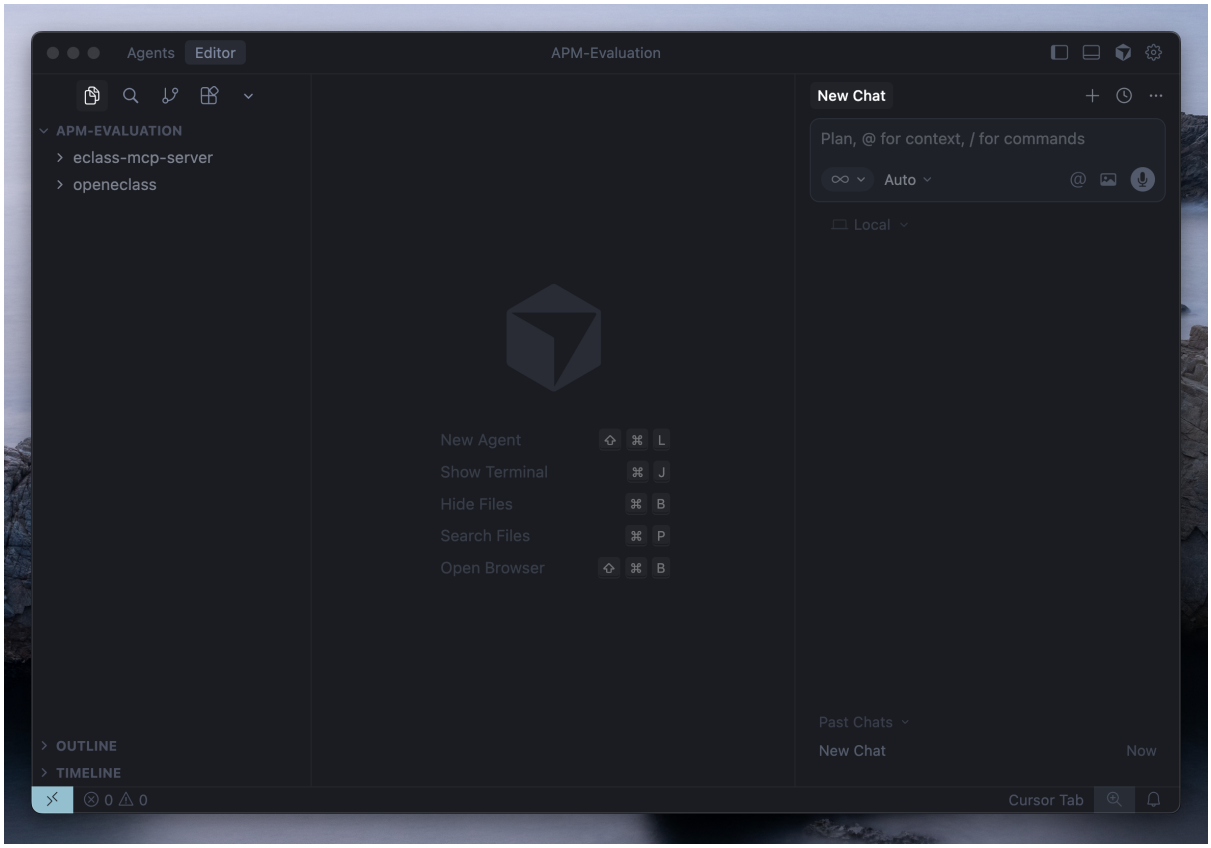
4. Session Workflow (Step-by-Step)

This section explains the session workflow in which you are to attempt to complete your task. Please follow these steps in order.

4.1. Ensure Environment is Set Up

Following Section 2.4, ensure you have installed Cursor and that you have correctly set up your environment with all dependencies installed and the required repositories.

Opening Cursor in your designated workspace directory would look like this:



Verify that all the base dependencies are installed by running:

```
/PATH/TO/YOUR/PYTHON3 --version   # Check Python version
uv --version                      # Check UV version
npm --version                     # Check NPM version (for APM)

# Check Python codebase dependencies (silent success, errors shown)
cd eclass-mcp-server
/PATH/TO/YOUR/PYTHON3 -c "import mcp"
/PATH/TO/YOUR/PYTHON3 -c "import requests"
/PATH/TO/YOUR/PYTHON3 -c "import bs4"
/PATH/TO/YOUR/PYTHON3 -c "import dotenv"
```

4.2. Session 1: Tool A

Initialize Git and Create Branch:

Ensure you're starting from a clean state and create an isolated branch:

```
cd eclass-mcp-server                # go to implementation codebase root
git checkout main                   # switch to main branch
git checkout -b tool-a-attempt      # create new branch for Tool A patch
```

Before you start working:

- Launch your first assigned AI tool (Tool A) following its setup instructions in Section 3.
- **Thoroughly read the** `Specification.md` (the PRD) provided alongside this guide.
- Use the **default task prompt** provided with the PRD to initiate your first chat.
- **Note the start time**; it will be used to track your request statistics and session duration.
- Use **only** your assigned Tool A for AI assistance during this session.
- Start the timer and get to work!

> **Important Note on Model Selection**
>
> For study validity, all participants **must** use **Cursor's Composer 1** model for the **entire duration** of both Tool A and Tool B sessions.

Work Until Completion or Time Limit:

- Continue working on the task for up to **3 hours**.
- **Stop at the 3-hour mark**, even if you haven't completed the task.
- Alternatively, stop when you have fully completed the task (if before 3 hours).

Create Code Patch:

After the 3 hour mark or upon task completion, generate your solution patch file for this session:

```
git add . && git commit -m "Tool A solution"          # commit changes
git diff main tool-a-attempt > tool_a_solution.patch  # create patch file
```

Export AI Interaction Logs:

Follow the specific instructions in Section 5 to export chat conversations and retrieve request statistics. Organize these in a `tool_a_logs/` folder.

Clean Up and Ensure Isolation:

Return to main branch and remove the solution branch:

```
git checkout main
git branch -D tool-a-attempt
```

Git Note

Feel free to use Git for version control during your session. However, ensure that you **do not modify the main branch directly**. All code changes related to your task should be committed to your isolated branch (`tool-a-attempt` for Tool A and `tool-b-attempt` for Tool B) before concluding the session.

Important: After creating the patch file, **immediately save/move it to a safe location outside your repository** (e.g., your desktop or a dedicated submission folder). **Do not** delete your branch until you have saved the patch file and exported all logs; these files will be lost if you delete the branch first.

Complete Tool A Survey:

Fill out the brief Tool A post-session survey.

Session 1 Checklist:

Before moving to Session 2, ensure you have completed the following:

- Created the patch file `tool_a_solution.patch` and deleted the `tool-a-attempt` branch.
- Collected AI interaction logs and organized them in `tool_a_logs/` folder:
  - Exported all chat conversations as `.md` files in `tool_a_chat_conversations/`.
  - Exported and filtered your token usage as `tool_a_request_stats.csv`.
- Completed the Tool A post-session survey.

4.3. Session 2: Tool B

Follow the same workflow as Session 1, but with the following adjustments:

- Create a branch named `tool-b-attempt` from the clean `main` branch.
- Use **Tool B** with **Cursor's Composer 1** model, following its setup instructions in Section 3.
- After the 3 hour mark or upon task completion, generate the patch file as `tool_b_solution.patch`:

```
git diff main tool-b-attempt > tool_b_solution.patch
```

- Export logs to `tool_b_logs/` folder following the same procedure as Tool A.
- Complete the Tool B post-session survey.

You may start Session 2 immediately after Session 1 or take a break. Ensure you are ready to approach Session 2 from scratch, maintaining a fresh perspective and minimizing any influence from Session 1.

> **Important Session 2 Note**
>
> To accurately track your interaction statistics for Tool B, please **make absolutely sure you note the start time at the beginning of Session 2**. This timestamp will help isolate the request data specific to this session.

Session 2 Checklist:

Before concluding your work, ensure you have completed the following:

- Created the patch file `tool_b_solution.patch` and deleted the `tool-b-attempt` branch.
- Collected AI interaction logs and organized them in `tool_b_logs/` folder:
  - Exported all chat conversations as `.md` files in `tool_b_chat_conversations/`.
  - Exported and filtered your token usage as `tool_b_request_stats.csv`.
- Completed the Tool B post-session survey.

5. Exporting AI Interaction Logs from Cursor

To export your AI interaction logs (chat conversations and AI request statistics) from Cursor, follow these steps for both sessions:

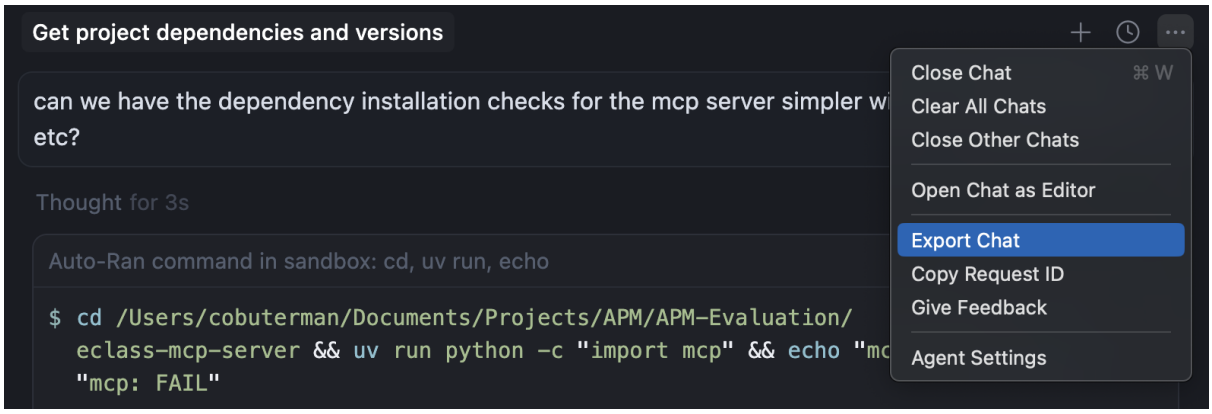
5.1. Export Chat Conversations

When using Tool A or Tool B, you may open multiple chat conversations in Cursor. For example, with APM, you will typically open at least three conversations: **Setup Agent**, **Manager Agent**, and at least one **Implementation Agent**. For Spec-kit or Plan Mode, the number of conversations may vary depending on your usage of these tools.

It is crucial to retain **all workflow conversations** for analysis. Do not delete any conversations during your sessions. If a conversation is accidentally deleted, please notify the researcher immediately.

To export each conversation, follow these steps:

- For each chat conversation opened during your session, navigate to that specific chat.
- In the top-right corner of the chat window panel, click on the **three-dot options menu**.
- From the dropdown menu, select the **"Export Chat"** option.



- Choose a location on your local machine to save the exported `.md` file.

Store and name the exported files appropriately and organize them as follows:

- Create a subfolder `tool_a_chat_conversations/` within your logs folder
- Save all chat conversation `.md` files in this subfolder (e.g., `tool_a_setup_agent.md`)
- Save the request statistics CSV directly in the logs folder as `tool_a_request_stats.csv`

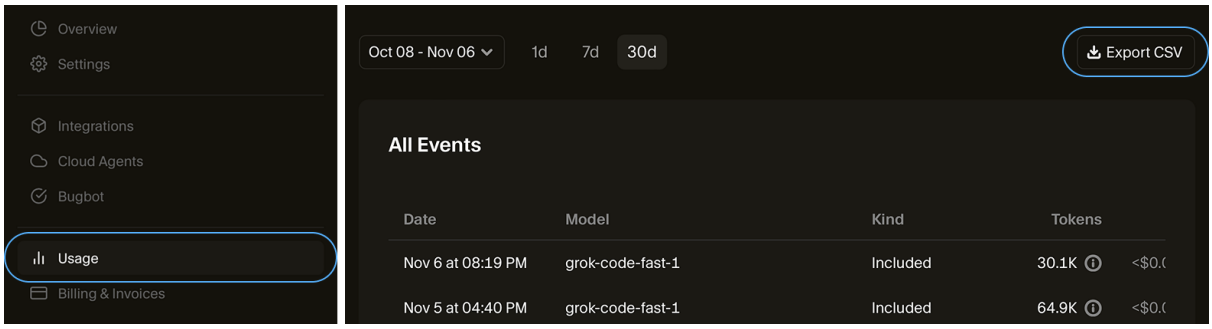
Important Chat Export Note

Consider exporting conversations **immediately after completion** during your session to avoid accidental deletions. For example, export APM's Setup Agent conversation right after the setup phase.

5.2. Retrieve AI Request Statistics

To retrieve your AI request statistics from Cursor, follow these steps:

- Login to your Cursor account and go to the Cursor Dashboard.
- On the sidebar, click on the **"Usage"** tab to access your usage statistics.
- Click on the **"Export CSV"** button located at the top-right corner of the Usage page.



This will immediately download a CSV file containing your AI request statistics to your local machine. This file contains AI requests made across all your sessions.

To isolate the requests made during your specific session, use the provided python script `filter_csv_by_datetime.py` with the **start time** you noted at the beginning of each session and rename the output file appropriately (e.g., `tool_a_request_stats.csv` for Tool A).

For example, if your session started on 2025-11-06 at 14:30 in the Europe/Athens timezone:

```
python3 filter_csv_by_datetime.py usage-events.csv 2025-11-06 14:30 Europe/Athens
```

Important Timing Note

This will create a filtered CSV file containing only requests from that point onward (e.g., `usage-events_filtered_2025-11-06_14:30_Europe_Athens.csv`).

Run the filtering script **immediately after completing your session** to ensure the CSV includes all requests from the session start up to your last AI request. If you wait too long before filtering, additional requests from other activities may be included.

5.3. Organize Logs for Submission

Save the exported chat conversations and filtered request statistics in a dedicated logs folder. For example, create a `tool_a_logs/` folder containing your chat exports and request statistics.

5.4. Log Organization Summary

After completing the above steps for both sessions, you will have organized logs folders:

- `tool_a_logs/` folder for Tool A
- `tool_b_logs/` folder for Tool B

These folders will be included in your final submission zip files alongside the patch files.

5.5. Create Submission Zip Files

Once you have your patch file and organized logs folder for each session, create the final submission zip files using your assigned participant ID:

```
# Replace {id} with your participant ID number
# For Tool A
zip -r id_{id}_tool_a_submission.zip tool_a_solution.patch tool_a_logs/

# For Tool B
zip -r id_{id}_tool_b_submission.zip tool_b_solution.patch tool_b_logs/
```

Important Submission Note

Ensure both the patch file and logs folder are at the root level of your zip file (not in subdirectories). The zip file should contain exactly two items: the `.patch` file and the `tool_X_logs/` folder. Use your assigned participant ID in the filename format `id_X_tool_X_submission.zip` as shown above.

6. Final Submission

After completing both sessions, submit all your data to our anonymous collection form.

Submission Checklist

Please be sure to upload the following single `.zip` files (one for each session):

```
id_{id}_tool_a_submission.zip    # Contains: tool_a_solution.patch + tool_a_logs/
id_{id}_tool_b_submission.zip    # Contains: tool_b_solution.patch + tool_b_logs/
```

7. Contact & Help

This study is conducted for a Bachelor's thesis by Kostantinos Chaidemenos - CobuterMan.

If you encounter any technical issues or have questions about the study procedure, contact:

```
sdi2200262@di.uoa.gr        # Email
cobuter_man                 # Discord
```

Thank you again for your time and contribution to this research.

